Start projektu – pierwsze zajęcia

Projekty zespołowe – laboratorium · co robimy na kick-offie i co trzeba przygotować na tydzień 2

1. Plan pierwszych zajęć (stacjonarnie, 90 min)

Czas	Punkt	Rezultat
0–15 min	Wprowadzenie: jak działa przedmiot, model 3 × 30 min na Teams, omówienie regulaminu	Wszyscy znają zasady i harmonogram.
15–30 min	Pokaz na żywo: od zadania w Jirze, przez gałąź Git i pracę z asystentem AI w terminalu, po Merge Request i pipeline w GitLabie	Wiadomo, jak ma wyglądać praca „na co dzień”.
30–45 min	Tworzenie zespołów (4–5 osób)	Lista składów.
45–60 min	Podział ról w zespołach: lider, osoba kontaktowa, tester, deweloperzy	Role zapisane na liście.
60–80 min	Burza mózgów: 2–3 pomysły na aplikację w każdym zespole, szybka konsultacja z prowadzącym	Kierunek tematu.
80–90 min	Omówienie zadań na tydzień 2 i ustalenie slotów na Teams	Grafik spotkań zdalnych.

2. Zadania na tydzień 2 – lista kontrolna

Wszystkie punkty muszą być gotowe **24 h przed pierwszym spotkaniem zespołu na Teams**. Osoba kontaktowa wysyła na kanale zespołu link do repozytorium, link do Jiry i potwierdzenie wykonania listy.

A. Każdy członek zespołu (indywidualnie)

- ☐ Konto w GitLabie i dodany klucz SSH; skonfigurowany Git (`git config --global user.name` / `user.email` – prawdziwe imię i nazwisko).
- ☐ Konto w Jirze i dołączenie do projektu zespołu.
- ☐ Zainstalowany asystent AI w terminalu (Claude Code lub OpenAI Codex CLI) i wykonane jedno proste zadanie w repozytorium testowym, np. „wyjaśnij strukturę tego projektu”.
- ☐ Przeczytane materiały z punktu 5.
- ☐ Jeden własny commit w repozytorium zespołu przez Merge Request – dopisanie siebie do tabeli zespołu w README.

B. Zespół (wspólnie)

- ☐ Repozytorium w GitLabie: prowadzący dodany, gałąź `main` chroniona, plik `.gitignore`, README v1.
- ☐ Projekt Jira typu Scrum: prowadzący dodany, backlog z min. **10 user stories** w formacie „Jako… chcę… aby…”, każda z kryteriami akceptacji.
- ☐ **Karta projektu** (punkt 3) w pliku `docs/karta-projektu.md`.
- ☐ Ustalona komunikacja wewnętrzna zespołu i stały termin spotkania roboczego bez prowadzącego.

3. Karta projektu – szablon

Karta projektu to „umowa” zespołu z prowadzącym. Po zatwierdzeniu na spotkaniu w tygodniu 2 stanowi podstawę oceny produktu. Objętość: 2–4 strony.

Sekcja	Co powinno się w niej znaleźć
1. Nazwa i opis	Nazwa robocza, opis w 3–5 zdaniach: jaki problem rozwiązuje aplikacja i dla kogo.
2. Zespół i role	Tabela: imię i nazwisko, rola, login GitLab. Uzasadnienie przydziału ról w 1–2 zdaniach.
3. Użytkownicy	Kim są użytkownicy (2–3 typy, tzw. persony) i jakie mają potrzeby.
4. Zakres funkcjonalny (MoSCoW)	**Must have** – minimum, bez którego projekt nie istnieje (to jest MVP na 7. tydzień). **Should have** – planowane, ważne. **Could have** – jeśli starczy czasu. **Won't have** – świadomie wyłączone z zakresu.
5. Wymagania niefunkcjonalne	Np. wydajność, bezpieczeństwo (logowanie, hasła), platforma (web / mobile / desktop), dostępność.
6. Stos technologiczny	Język, framework, baza danych, biblioteka testowa, sposób wdrożenia. Krótko uzasadnij wybór.
7. Architektura (szkic)	Prosty diagram komponentów: frontend, backend, baza, usługi zewnętrzne.
8. Strategia testów	Przygotowuje tester: jakie poziomy testów, jakie narzędzia, co wchodzi do pipeline.
9. Ryzyka	3–5 ryzyk (np. „nikt nie zna frameworka X”) i plan, jak je ograniczyć.
10. Plan sprintów	Wstępne przypisanie funkcji „Must have” do sprintów 1–3.
11. Zasady zespołu	Definition of Done, zasady code review, konwencja gałęzi i commitów, kto i jak korzysta z AI.

4. Wymagania wobec tematu

- Aplikacja ma **realnego użytkownika** i rozwiązuje konkretny problem (nie „sklep internetowy ogólnie”).
- Musi zawierać co najmniej: trwałe przechowywanie danych (baza), logikę biznesową wartą przetestowania oraz interfejs użytkownika lub publiczne API.
- Zakres „Must have” musi być wykonalny w ok. 6 tygodni pracy zespołu – lepiej mniejszy, ale dopracowany.
- Technologia dowolna, ale cały zespół musi być w stanie w niej pracować. Każdy zespół ma inny temat – w razie kolizji decyduje kolejność zgłoszenia.

Przykładowe kierunki: system rezerwacji sal lub sprzętu na wydziale · aplikacja do podziału kosztów w grupie znajomych · tracker nawyków lub treningów z wykresami · system zgłaszania usterek w akademiku · planer posiłków z listą zakupów · giełda korepetycji studenckich · aplikacja do zarządzania biblioteczką gier planszowych z wypożyczeniami.

5. Z czym się zapoznać przed tygodniem 2

Temat	Materiał	Dla kogo
Git	*Pro Git* (wersja polska na git-scm.com/book/pl/v2) – rozdziały 1–3: podstawy, gałęzie	wszyscy
Merge Requesty	Dokumentacja GitLab: Merge requests oraz Protected branches (docs.gitlab.com)	wszyscy, szczególnie lider

Temat	Materiał	Dla kogo
Konwencja commitów	conventionalcommits.org – specyfikacja (5 min czytania)	wszyscy
Jira i Scrum	Atlassian: samouczek „How to do scrum with Jira” oraz pisanie user stories	lider, osoba kontaktowa
CI/CD	GitLab CI/CD – „Tutorial: Create and run your first GitLab CI/CD pipeline” (docs.gitlab.com)	lider, tester
Testy	Dokumentacja biblioteki testowej dla wybranego języka (np. pytest, JUnit, Jest, GoogleTest)	tester, deweloperzy
AI w terminalu	Dokumentacja Claude Code (docs.claude.com) lub OpenAI Codex CLI (github.com/openai/codex) – instalacja, pierwsze kroki, plik instrukcji projektu	wszyscy
README	makeareadme.com – struktura dobrego README	osoba kontaktowa

6. Jak będzie wyglądać pierwsze spotkanie na Teams (tydzień 2)

1. Osoba kontaktowa udostępnia ekran i w 5 minut przedstawia Kartę projektu.
2. Wspólnie przechodzimy przez zakres MoSCoW – tu najczęściej **tniemy zakres**, żeby był wykonalny.
3. Sprawdzam repozytorium, ochronę gałęzi, backlog w Jirze i commity każdej osoby.
4. Zatwierdzam temat lub wskazuję poprawki (termin poprawek: 3 dni).
5. Ustalamy cel Sprintu 0 (tydzień 3): szkielet aplikacji + pierwszy działający pipeline CI.

> **Najczęstsze błędy na starcie:** zbyt duży zakres („zrobimy drugiego Facebooka”), jedna osoba robiąca wszystko w repozytorium, brak kryteriów akceptacji w user stories, wybór technologii, której nie zna większość zespołu, oraz commitowanie bezpośrednio do `main`.